tapis2pdf

Joe Stubbs

Mar 20, 2025

INTRODUCTION:

1 Getting Started **3**
- 1.1 Getting Ready . . . 4
- 1.2 Tapis Quickstart . . . 4

2 Technical Guide - Overview **13**

3 Sites, Tenancy and Authentication **15**
- 3.1 Sites . . . 15
- 3.2 Tenants . . . 16
- 3.3 Authentication . . . 17

Add your content using `reStructuredText` syntax. See the reStructuredText documentation for details.

CHAPTER

ONE

GETTING STARTED

This Getting Started guide will walk you through the initial steps of setting up the necessary accounts and installing the required software before moving to the Tapis Quickstart. If you are already using Docker Hub and the TACC Cloud APIs, feel free to jump right to the *Tapis Quickstart* or check out the Tapis Live Docs site.

- *Getting Ready*
  - *cURL or Python: Pick Your Path*
  - *Amazon S3*
  - *A TACC Account*
- *Tapis Quickstart*
  - *Python Example*
    - *Getting a Tapis Token*
    - *Register the S3 Storage with Tapis*
    - *Create an Access Credential*
    - *Access the files in our S3 Bucket*
  - *cURL Example*
    - *Getting a Tapis Token*
    - *Register the S3 Storage with Tapis*
    - *Create an Access Credential*
    - *Access the files in our S3 Bucket*
  - *Sentiment-Analysis Application Tutorial*
    - *Create A Tapis Token*
    - *Create a system*
    - *Create An Application*
    - *Application Arguments*
    - *Submitting a job*
    - *Jobs List*
    - *Jobs Output*

* *Next Steps*

1.1 Getting Ready

1.1.1 cURL or Python: Pick Your Path

This guide provides examples in both the command line web interface *cURL* and the Python SDK. cURL is a command line tool for transferring data via URLs and receiving responses; cURL is available in your terminal (for MacOS, Linux, and Unix users), and transfers data from or to a server using one of a number of internet protocols. You could also choose to use a tool like Postman which which provides a nice GUI on top of cURL.

To run the Python examples you will need a functioning Python3 environment and the Tapis Python APIs installed on your system (read more about the tapispy project at PyPi).

1.1.2 Amazon S3

In either case you will be accessing an Amazon AWS S3 storage bucket in this Tapis example. In the code we assume you have created an S3 bucket with URL *https://<your_bucket_name>.s3.amazonaws.com/* and that you have access to the bucket via an AWS user identity with `accessKey` and `accessSecret`. If you are new to Amazon's S3 service, you can get started for free to complete this tutorial. Check out their Getting Started Guide and follow the tutorial through *Step 2: Upload an object to your bucket*. Our example will list the files stored in this bucket, so you may want to upload a file to your bucket so you can verify that your code works; a small image file would do the trick. In the examples below we have created an S3 bucket called *tapisbucket* and uploaded a single file to it called *test_image.jpg*.

1.1.3 A TACC Account

The main instance of the Tapis platform is hosted at the Texas Advanced Computing Center (TACC). TACC designs and deploys some of the world's most powerful advanced computing technologies and innovative software solutions to enable researchers to answer complex questions. To follow along with this Quickstart and use the TACC-hosted Tapis platform, please create a TACC account .

1.2 Tapis Quickstart

In this guide, we will use the Tapis APIs to list files on the Amazon AWS S3 storage system using the *tacc* instance of Tapis (an instance of Tapis is called a **tenant**; since Tapis is open source any organization can install it and run their own tenant) with base URL *https://tacc.tapis.io*. We will work the example in two ways: using cURL from the command line and using a Python script. Pick the path that most closely matches your needs.

1.2.1 Python Example

Getting a Tapis Token

To begin we use our TACC account credentials to get a Tapis token from the authenticator. A valid token is needed to interact with Tapis. You will note that your token is returned with an expiration date and time. If you want to continue to use Tapis after your token expires, you will need to get a new one following the same steps.

For the purpose of this tutorial, we will be using the base_url of `https://tacc.tapis.io`. To discover other tenants, and find the details specific to that tenant, such as owner, base_url, tenant_id, etc. you can use following curl command in your terminal window.

```
curl https://admin.tapis.io/v3/tenants | jq
```

Note that `jq` is a command-line JSON formatter. Checking out their download instructions.

From here, you will use the `ctrl-F` function in your terminal window to search for your base_url. A note of caution, be sure to verify if you are using the DEV or PROD tenants; DEV tenants will be labeled with a delineation of -dev and noted in the description as such.

For more information about tenant listing and search techniques please see the Tenancy, Sites and Authentication.

In Python code below you will replace *your_tacc_username* and *your_tacc_password* with your TACC username and password, preserving the quotation marks shown in the command below. See more about the Tapis Python SDK, `tapipy`, including how to install it, from here.

This call does not produce output. However you can use the following code to see the access token you just created.

```
t.access_token
```

The output should look similar to the following; it describes the access token that was just created.

```
access_token: *very long string of alphanumeric characters*
claims: {'jti': '007fa9e6-f044-4817-a812-12292b2bdbe3', 'iss': 'https://tacc.tapis.io/v3/
↪tokens', 'sub': 'your_tacc_username', 'tapis/tenant_id': 'tacc', 'tapis/token_type':
↪'access', 'tapis/delegation': False, 'tapis/delegation_sub': None, 'tapis/username':
↪'your_tacc_username', 'tapis/account_type': 'user', 'exp': 1657686889, 'tapis/client_id
↪': None, 'tapis/grant_type': 'password'}
expires_at: 2022-07-13 04:34:49+00:00
expires_in: <function Tapis.add_claims_to_token.<locals>._expires_in at 0x10a070280>
jti: 007fa9e6-f044-4817-a812-12292b2bdbe3
original_ttl: 14400
```

Where you will have your own access token and the placeholder *your_tacc_username* will be replaced with the username you used.

Register the S3 Storage with Tapis

Now that we have an access token, we are ready to create a Tapis `system` object. Remember that the Tapis APIs are a framework for accessing a wide variety of computational resources: for this example, we will register the S3 bucket we pre-created with Amazon's AWS S3 service (if this is unfamiliar to you, refer back to the *Getting Ready* section above). This step registers the S3 bucket with Tapis so it can access the bucket on your behalf. Note that the value of `id` needs to be unique within the Tapis tenant you are using; for this example we show the string *your_tapis_system_id*, but you'll need to choose your own globally (within the Tapis tenant you are using) unique id (possibly including your user name, for example, or some other unique id; for example, something like *username.tapis_test_v1*). The values for `host` and `bucketName` are set based on the values you used when you set up your own AWS S3 bucket, as discussed above. In this example they are

```
# To register the S3 bucket with Tapis
# the description of an S3 bucket
s3_bucket = {
  "id":"your_tapis_system_id",
  "description":"Tapis Test Bucket",
  "host":"tapisbucket.s3.amazonaws.com",
  "systemType":"S3",
  "defaultAuthnMethod":"ACCESS_KEY",
  "bucketName":"tapisbucket",
  "canExec": False,
}
```

(continues on next page)

(continued from previous page)

```
# create the system in Tapis
t.systems.createSystem(**s3_bucket)
```

The output of the command will show the URL to the system you just created in the Tapis tenant.

Create an Access Credential

With our system created and linked to the S3 storage bucket, we need to create an access credential for Tapis to access our bucket on our behalf. Note that in this example we have created an access key and secret for the IAM user in our AWS instance; when substituting your own access key and secret into the code below, remember to preserve the single quotation marks enclosing these pieces of data.

```
t.systems.createUserCredential(systemId='your_tapis_system_id',
                               userName='your_tacc_username',
                               accessKey='IAM user access key that you created in the␣
↪AWS interface',
                               accessSecret='access secret from the AWS interface for␣
↪the key you created')
```

The output of the command should look similar to the text below.

```
{'result': None,
'status': 'success',
'message': 'SYSAPI_CRED_UPDATED Credential updated. jwtTenant: tacc jwtUser: your_tacc_
↪username OboTenant: tacc OboUser: your_tacc_username System: your_tapis_system_id␣
↪User: your_tacc_username',
'version': '1.2.1',
'metadata': None}
```

Access the files in our S3 Bucket

We are now able to list files in our bucket using the Files API (you can also use the API to add new files, delete files, rename files, and so on; but since this is a example, we've just selected a file listing as the example operation).

```
t.files.listFiles(systemId="your_tapis_system_id", path="/")
```

The output should include a list of all files in the bucket. For this example we only put one file in our bucket, test_image.jpg. The listing returns the name of that file along with some metadata:

```
[
group: None
lastModified: 2022-05-20T19:24:24Z
mimeType: image/jpg
name: test_image.jpg
nativePermissions: None
owner: None
path: /test_image.jpg
size: 87060
type: file
url: tapis://your_tapis_system_id/test_image.jpg]
```

1.2.2 cURL Example

As with the Python example, we will use the Tapis APIs to list files on the Amazon AWS S3 storage system using the *tacc* tenant with base URL *https://tacc.tapis.io*.

Getting a Tapis Token

To begin we use our TACC account credentials to get a Tapis token from the authenticator. A valid token is needed to interact with Tapis. You will note that your token is returned with an expiration date and time. If you want to continue to use Tapis after your token expires, you will need to get a new one following the same steps.

Type the curl command below into your terminal window, replacing *your_tacc_username* and *your_tacc_password* with your TACC user name and password, preserving the quotation marks.

```
$ curl -H "Content-type: application/json" -d '{"username": "your_tacc_username
↪", "password": "your_tacc_password", "grant_type": "password" }' https://
↪tacc.tapis.io/v3/oauth2/tokens
```

The output of this operation will look like the following (line breaks have been added for clarity in reading; your response string will not have line breaks); the phrase <*your access token string will be here*> will be replaced with your token access string, which is an 834 character alphanumeric string. Notice the expiration time in the return string.

```
{“message”:”Token created successfully.”,
 ”metadata”:{},
 ”result”:{“access_token”:{“access_token”:”<your access token string will be here>”,
    ”expires_at”:”2022-05-05T19:53:03.801252+00:00”,
    ”expires_in”:14400,”jti”:”8ef1d271-b923-49af-b2dd-ae05cc5da1ed”}},
 ”status”:”success”,
 ”version”:”dev”}
```

To work through the rest of the examples in this guide, you will need to add the token from the curl command to your environment using the variable name `JWT`. The example below shows how I added it to zsh (a bash variant); the precise method may vary with your shell:

```
$ export JWT=your_access_token_string
```

Register the S3 Storage with Tapis

Now that we have an access token, we are ready to create a Tapis `system` object. Remember that the Tapis APIs are a framework for accessing a wide variety of computational resources: for this example, we will register the S3 bucket we pre-created with Amazon's AWS S3 service (if this is unfamiliar to you, refer back to the *Getting Ready* section above). This step registers the S3 bucket with Tapis so it can access the bucket on your behalf. Note that the value of `id` needs to be unique within the Tapis tenant you are using; for this example we show the string *your_tapis_system_id*, but you'll need to choose your own globally (within the Tapis tenant you are using) unique id (possibly including your user name, for example, or some other unique id; for example, something like *username.tapis_test_v1*, but you'll need to select your own). The values for `host` and `bucketName` are set based on the you used when you set up your own AWS S3 bucket, as discussed above.

To keep the cURL command (relatively) readable, you first need to create a file in your path with the details of your S3 storage and Tapis system formatted as a JSON object; in this example we use the filename *system_s3.json*, with the following contents:

```
{
  "id":"your_tapis_system_id",
  "description":"Tapis cURL Test Bucket",
  "host":"tapisbucket.s3.amazonaws.com",
```

(continues on next page)

(continued from previous page)

```
  "systemType":"S3",
  "defaultAuthnMethod":"ACCESS_KEY",
  "bucketName":"tapisbucket",
  "canExec": False
}
```

Then you'll execute the following cURL command, being sure to specify the name of the file you created if you chose a different name:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems -d @system_s3.json
```

The output of the command will show the URL to the system you just created in the Tapis tenant.

```
{
  "result": {
    "url": "http://tacc.tapis.io/v3/systems/your_tapis_system_id"
  },
  "status": "success",
  "message": "SYSAPI_CREATED New system created. jwtTenant: tacc jwtUser: your_tacc_
↪username OboTenant: tacc OboUser: your_tacc_username System: your_tapis_system_id",
  "version": "1.2.3",
  "metadata": null
}
```

Create an Access Credential

With our system created and linked to the S3 storage bucket, we need to create an access credential for Tapis to access our bucket on our behalf. Note that in this example we have created an access key and secret for the IAM user in our AWS instance; when substituting your own access key and secret into the code below, remember to preserve the single quotation marks enclosing these pieces of data.

Again, to keep the cURL command (relatively) readable, you first need to create a file in your path with the details of your S3 access key formatted as a JSON object; in this example we use the filename cred_tmp.json, with the following contents:

```
{
  "accessKey":"IAM user access key that you created in the AWS interface",
  "accessSecret":"access secret from the AWS interface for the key you created"
}
```

Then you'll execute the following cURL command, being sure to specify the name of the file you created if you chose a different name:

```
$curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems/credential/your_tapis_system_id/user/your_tacc_username -d @cred_
↪tmp.json
```

The output of the command will show will look similar to that below.

```
{
  "result": null,
  "status": "success",
```

(continues on next page)

(continued from previous page)

```
  "message": "SYSAPI_CRED_UPDATED Credential updated. jwtTenant: tacc jwtUser: your_tacc_
↪username OboTenant: tacc OboUser: your_tacc_username System: your_tapis_system_id␣
↪User: your_tacc_username",
  "version": "1.2.3",
  "metadata": null
}
```

Access the files in our S3 Bucket

We are now able to list files in our bucket using the Files API. If you look closely at the URL you'll see that we are using the *files* access point – this URL returns a listing of the files in your bucket along with some metadata. Recall that for our example the bucket has a single image in it.

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/ops/your_tapis_system_id/
```

The output of the command will show will look similar to that below, where the path and file name will reflect how you set up your own S3 bucket and the file(s) you put in it. We added line breaks to the output below for readability.

```
{
  "status":"success",
  "message":"ok",
  "result":
  [
    {
      "mimeType":"image/jpg",
      "type":"file",
      "owner":null,
      "group":null,
      "nativePermissions":null,
      "url":"tapis://your_tapis_system_id/test_image.jpg",
      "lastModified":"2022-05-20T19:24:24Z",
      "name":"test_image.jpg",
      "path":"/test_image.jpg",
      "size":87060
    }
  ],
  "version":"1.2.2","metadata":{}
}
```

1.2.3 Sentiment-Analysis Application Tutorial

For those of you that want to dive straight into Tapis and begin to explore it's possibilites, this is our tutorial. In this tutorial you will create a system, sentiment-analysis application, and run a job with it. For the sake of brevity and speed, we will complete this tutorial utilizing the Python SDK, Tapipy.

Pre-requisites: Active TACC account, Tapipy installed, Access to Stampede3, Frontera, or non-MFA system.

Create A Tapis Token

Next you will need to gather your access token.

```
t.access_token
```

The output should look similar to the following; it describes the access token that was just created.

```
access_token: *very long string of alphanumeric characters*
claims: {'jti': '007fa9e6-f044-4817-a812-12292b2bdbe3', 'iss': 'https://tacc.tapis.io/v3/
↪tokens', 'sub': 'your_tacc_username', 'tapis/tenant_id': 'tacc', 'tapis/token_type':
↪'access', 'tapis/delegation': False, 'tapis/delegation_sub': None, 'tapis/username':
↪'your_tacc_username', 'tapis/account_type': 'user', 'exp': 1657686889, 'tapis/client_id
↪': None, 'tapis/grant_type': 'password'}
expires_at: 2022-07-13 04:34:49+00:00
expires_in: <function Tapis.add_claims_to_token.<locals>._expires_in at 0x10a070280>
jti: 007fa9e6-f044-4817-a812-12292b2bdbe3
original_ttl: 14400
```

Where you will have your own access token and the placeholder *your_tacc_username* will be replaced with the username you used.

Create a system

Next you will need to create a system. Your system must be hosted on a machine that you can SSH to. There are a variety of authentication methods such as PASSWORD, PKI_KEYS (SSH Keys: Private, Public pair), ACCESS_KEYS (S3), and TOKENS (GLOBUS). If you are using PKI_KEYS please be aware that they will only work if MFA is NOT enabled on that system. Also, you must place the public key on that system. The Public and Private key on your system and the Public key on the host system must be formatted for one line.

```
  system_def = {
  “id”: “<YOUR_SYSTEM_ID>”,
  "description": "test system",
  "systemType": "LINUX",
  "host”:”<HOST>,
  "defaultAuthnMethod": "PASSWORD",
  "rootDir": "/",
  "canExec": True,
  "jobRuntimes": [ { "runtimeType": "SINGULARITY" } ],
  "jobWorkingDir": "workdir",
}

t.systems.createSystem(**system_def)
```

When defining a HOST, it’s important to remember that it should be defined by the URL without the “https://”

This will return:

```
url: http://tacc.tapis.io/v3/systems/<YOUR_SYSTEM_ID>
```

Create An Application

Now that you have a system to run your jobs, you must create an application. Here is an example of an application definition:

```
  app_def = {
    "id": <app_id>,
    "version": "0.2",
    "description": "Application utilizing the sentiment analysis model from Hugging Face.
↪",
    "jobType": "FORK",
    "runtime": "DOCKER",
```

(continues on next page)

(continued from previous page)

```
    "containerImage": "tapis/sentiment-analysis:1.0.0",
    "jobAttributes": {
        "parameterSet": {
            "archiveFilter": {
                "includeLaunchFiles": False
            }
        },
        "memoryMB": 1,
        "nodeCount": 1,
        "coresPerNode": 1,
        "maxMinutes": 10
    }
}
```

With a system now created, we need to register this application to make it accessible on this tenant.

```
t.apps.createAppVersion(**app_def)
```

Application Arguments

With appArgs parameter you can specify one or more command line arguments for the user application. Arguments specified in the application definition are appended to those in the submission request. Metadata can be attached to any argument:

```
# Modify Job submission arguments
  pa = {
      "parameterSet": {
      "appArgs": [
              {"arg": "--sentences"},
              {"arg": "\"This is great\" \"This is not fun\""}
          ]
      }}
```

Submitting a job

Running a job only requires 3 items: a Job name (name), an app_id, and the app_id version. If you have not specified the Execution System in your application, you will need to specify it when submitting a job.

A simple submission would look like this:

```
job_response = t.jobs.submitJob(
    name='sentiment analysis',
    appId=app_id,appVersion='0.2',
    execSystemId=system_def,
    **pa #reintroducing modified job submission arguments
  )
```

All of the Job Submission Parameters can be found here Job Submission Parameters.

Everytime a job is submitted, a unique job_id (uuid) is generated. We will use this job_id with tapipy to get the job status and download the job output.

```
  # Get job uuid from the job submission response
print("*************************************************")
job_uuid = job_response.uuid
print("Job UUID: " + job_uuid)
print("*************************************************")
```

Jobs List

When you do a jobs list now, you can see your jobUuid.

```
t.jobs.getJobList()
```

Jobs Output

To download the output of a job you need to give it the jobUuid and output path. You can download a directory in the jobs' outputPath in zip format. The outputPath is relative to archive system specified.

```
  # Download output of the job
print("Job Output file:")

print("*************************************************")
jobs_output = t.jobs.getJobOutputDownload(jobUuid=job_uuid,outputPath='stdout')
print(jobs_output)
print("*************************************************")
```

Next Steps

This concludes the Sentiment-Analysis tutorial. This tutorial was presented at PEARC'24. Please feel free to explore and read more of the Tapis documentation. If you would like to see more recent Tapis tutorials, please see Tapis Tutorials.

CHAPTER

TWO

TECHNICAL GUIDE - OVERVIEW

The Technical Guide for Tapis provides a detailed reference to the primary services in the platform.

- *Sites, Tenancy and Authentication*: Complete reference for Tapis authentication.
- systems: Registering and working with remote storage and execution resources.
- files: Managing data on Tapis systems.
- apps: Registering and working with applications.
- jobs: Running an application on a system.
- meta: Storing and retrieving metadata pertaining to your projects.
- pgrest: Hosted, HTTP RESTful API to a managed Postrges database instance.
- pods: Create long-lived network enabled pods for databases or servers.
- streams: Managing streaming data from sensors and other instruments.
- actors: Execute containerized functions ("actors") in response to events or other messages.
- security: Securing the digital assets comprising your project.
- workflows: Create and execute workflows in the Tapis ecosystem.
- sharing: Sharing Tapis resources with colleagues.
- pythondev: Tapipy and Tapisservice usage, special attributes, and examples for Python development.

CHAPTER

THREE

SITES, TENANCY AND AUTHENTICATION

3.1 Sites

Tapis supports geographically distributed deployments where different components are running in different data centers and managed by different institutions. These physically isolated installations of Tapis software are referred to as *sites*. There is a single *primary site* and zero or more *associate sites* within a Tapis installation.

3.1.1 Primary Site

The primary site in a Tapis installation runs a complete set of Tapis API services and all associated 3rd-party services, such as databases and message queues. The creation of new sites is coordinated through the primary site, and the primary site runs the unique instance of the Sites and Tenants API (see *Tenants* below) which maintain the complete registry of all sites and tenants in the installation.

The primary site of the main Tapis installation is hosted at the Texas Advanced Computing Center at the tapis.io domain.

3.1.2 Associate Sites

Associate sites are required to run the Tapis Security Kernel, a compliant Token Generator API, and one or more additional Tapis services. Each associate site is managed and operated by a separate, partner institution. For Tapis services not run at the associate site, the corresponding service at the primary site is used for requests. In this way, partner institutions can choose which Tapis services to run within their institution and leverage the primary site deployment for the rest.

3.1.3 Deployment

The official Tapis deployment tooling targets the Kubernetes container orchestration platform. The project maintains a set of deployment templates which can be used in conjunction with configuration files to deploy Tapis services. If your institution is interested in becomming a Tapis associate site please contact us.

Details about the current list of sites is available from the tenants API. For example, one can retrieve the full list of sites as follows:

With PySDK:

```
>>> t.tenants.list_sites()
```

With CURL:

```
$ curl https://admin.tapis.io/v3/sites
```

The response will look similar to the following (the response below is truncated for brevity):

```
[
 base_url: https://tapis.io
 primary: True
 services: ['systems', 'files', 'security', 'tokens', 'streams', 'authenticator', 'meta',
↪ 'actors']
 site_admin_tenant_id: admin
 site_id: tacc
 tenant_base_url_template: https://${tenant_id}.tapis.io]
 .  .  .
]
```

Each site has a `site_id` as well as a list of Tapis services it provides and the tenant ID of the administrative tenant (`admin_tenant`) associated with it.

3.2 Tenants

Tapis is a *multi-tenant* platform, meaning that different projects (or *tenants*) can have logically isolated views of the Tapis objects (i.e., the systems, files, actors, etc.) they create for their project.

Each tenant is made up of the following:

1. A base URL with which to access the tenant; by default, the base URL takes the form `https://<tenant_id>.tapis.io` where `tenant_id` is a short, unique identifier for the tenant in the Tapis system. For example, `https://tacc.tapis.io` is the base URL for the `tacc` tenant.
2. An *authenticator* providing the rules for who can authenticate in the tenant.

Additionally, each tenant is "managed" by a site.

To see the current list of tenants registered with Tapis, we can use the tenants API.

With PySDK:

```
>>> t.tenants.list_tenants()
```

With CURL:

```
$ curl https://tacc.tapis.io/v3/tenants
```

The response will look similar to the following (the response below is truncated for brevity):

```
allowable_x_tenant_ids: ['tacc']
authenticator: https://tacc.tapis.io/v3/oauth2
base_url: https://tacc.tapis.io
create_time: Thu, 02 Jul 2020 23:45:16 GMT
description: Production tenant for all TACC users.
is_owned_by_associate_site: False
last_update_time: Thu, 02 Jul 2020 23:45:16 GMT
owner: CICSupport@tacc.utexas.edu
public_key: -----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAz7rr5CsFM7rHMFs7uKIdcczn0uL4ebRMvH8pihrg1tW/
↪fp5Q+5ktltoBTfIaVDrXGF4DiCuzLsuvTG5fGElKEPPcpNqaCzD8Y1v9r3tfkoPT3Bd5KbF9f6eIwrGERMTs1kv7665pliwehz91n
↪EnSg1N3Oj1x8ktJPwbReKprHGiEDlqdyT6j58l/I+9ihR6ettkMVCq7Ho/
↪bsIrwm5gP0PjJRvaD5Flsze7P4gQT37D1c5nbLR+K6/T0QTiyQIDAQAB
-----END PUBLIC KEY-----
security_kernel: https://tacc.tapis.io/v3/security
```

(continues on next page)

(continued from previous page)

```
service_ldap_connection_id: None
tenant_id: tacc
token_service: https://tacc.tapis.io/v3/tokens
user_ldap_connection_id: tacc-all,

allowable_x_tenant_ids: ['dev']
authenticator: https://dev.tapis.io/v3/oauth2
base_url: https://dev.tapis.io
create_time: Fri, 19 Jun 2020 20:36:38 GMT
description: The dev tenant.
is_owned_by_associate_site: False
last_update_time: Fri, 19 Jun 2020 20:36:38 GMT
owner: CICSupport@tacc.utexas.edu
public_key: -----BEGIN PUBLIC KEY-----

. . .
```

Here we see the first two tenants registered in the Tapis framework, the `tacc` and `dev` tenants.

In general, the rules for authentication vary from tenant to tenant within Tapis. For example, in the `tacc` tenant, any user with a valid TACC account can authenticate and use the APIs. The `dev` tenant is a development sandbox with test accounts used by the core Tapis team.

This documentation focuses on the `tacc` tenant; however, much of what follows in the subsequent sections will be the same regardless of the tenant you are using.

3.3 Authentication

The default authenticator provided by the Tapis project is based on OAuth2, and this is the authentication mechanism in place for the `tacc` tenant. The OAuth2-based authentication services are available via the `/v3/oauth2` endpoints.

OAuth uses different *grant type flows* for generating tokens in different situations. We do not provide a comprehensive guide to OAuth2; for that, we refer the reader to the OAuth2 docs. Instead, we provide a guide to the two most common use cases for users: generating tokens for themselves using the *password* grant flow, and generating tokens on behalf of others in a web application using the *authorization code* grant flow.

In the PySDK examples that follow, we will tacitly assume the `tapipy.tapis.Tapis` object has been instantiated as the Python object `t`. There are several options in the `Tapis` constructor, but the basic options include `base_url` and `username`, for example:

```
>>> t = Tapis(base_url='https://tacc.tapis.io', username='jdoe')
```

3.3.1 Password Grant - Generating a Token For Yourself

The simplest case is that you want to generate a Tapis OAuth token for yourself; to do this you can use the *password* grant flow, providing your username and password.

Tapis v3 tries to make this process as easy as possible by providing a simplified version of the password grant flow that does not require an OAuth client (see the *OAuth Clients* section).

With PySDK:

```
>>> t = Tapis(base_url='https://tacc.tapis.io', username='apitest', password='abcd123')
>>> t.get_tokens()
```

With CURL:

```
> curl -H "Content-type: application/json" -d '{"username": "apitest", "password":
↪"abcde123", "grant_type": "password" }' \
https://tacc.tapis.io/v3/oauth2/tokens
```

In the PySDK, the access token is a first-class Python object stored on the Tapis object (the `t` in the examples above). We can inspect it

```
>>> t.access_token

access_token: eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.
↪eyJqdGkiOiJmN2I5YjE5ZS02MDk5LTRmODItYTcyMi1iNjEwYzVkMGJhMGMiLCJpc3MiOiJodHRwczovL3RhY2MudGFwaXMuaW8vd
↪alC8rRM-zNsHKcUiz3-tOJPaYtFksKb4Bit_aFE1HH_X_znnP2QkJaqc-
↪xaRoMlQu26MN72TlJE0siIN3T38xXWBGDumHUYbvnNzT-
↪7lk7AQU5MHSyCWx8IRDmTSbqmWOG8WBMCIV9Dh84mDd-X6eLJQ_cz1QqMAiI_cPgA9VVE22qDK3Lbz2pp9t0sm-
↪l9XjE5y5Im8Y0B2p0ssPD0TjW20C5yngZ4-4jowDafboKlscog9ko-adrsVIjG_r-
↪ccCUX3r8SVwQLypZFZAPKqbVzl8jt_
↪mCi30W8AYwiaYGmH7INBbHI9hO7kwJNFMuSylejFhMslxgdzGlIAyXauwg
claims: {'jti': 'f7b9b19e-6099-4f82-a722-b610c5d0ba0c', 'iss': 'https://tacc.tapis.io/v3/
↪tokens', 'sub': 'apitest@tacc', 'tapis/tenant_id': 'tacc', 'tapis/token_type': 'access
↪', 'tapis/delegation': False, 'tapis/delegation_sub': None, 'tapis/username': 'apitest
↪', 'tapis/account_type': 'user', 'exp': 1595099456, 'tapis/client_id': None, 'tapis/
↪grant_type': 'password'}
expires_at: 2020-07-18 19:10:56+00:00
expires_in: <function Tapis.set_access_token.<locals>._expires_in at 0x7f29e213c510>
jti: f7b9b19e-6099-4f82-a722-b610c5d0ba0c
original_ttl: 14400
```

What we see is that the `access_token.access_token` is a Python string representing a JSON Web Token (JWT). JWTs are cryptographically signed with the private key associated with the tenant, and anyone can validate the signature by using the corresponding public key associated with the tenant (see Tenants section above). The public key for each tenant is available from the Tenants API. The core Tapis services will validate the access token sent on a given API call using the public key associated with the tenant to verify the JWT signature.

3.3.2 Using a Token

In order to use an access token in an API request to Tapis, pass the token in as the value of the `X-Tapis-Token` header. The PySDK will automatically send the token via this header for you. In CURL examples used throughout this documentation, we assume the raw JWT string representing an access token (like the above) has been exported as a shell variable; i.e.,

```
$ export JWT=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.
↪eyJqdGkiOiJmN2I5YjE5ZS02MDk5LTRmODItYTcyMi1iNjEwYzVkMGJhMGMiLCJpc3MiOiJodHRwczovL3RhY2MudGFwaXMuaW8vd
↪alC8rRM-zNsHKcUiz3-tOJPaYtFksKb4Bit_aFE1HH_X_znnP2QkJaqc-
↪xaRoMlQu26MN72TlJE0siIN3T38xXWBGDumHUYbvnNzT-
↪7lk7AQU5MHSyCWx8IRDmTSbqmWOG8WBMCIV9Dh84mDd-X6eLJQ_cz1QqMAiI_cPgA9VVE22qDK3Lbz2pp9t0sm-
↪l9XjE5y5Im8Y0B2p0ssPD0TjW20C5yngZ4-4jowDafboKlscog9ko-adrsVIjG_r-
↪ccCUX3r8SVwQLypZFZAPKqbVzl8jt_
↪mCi30W8AYwiaYGmH7INBbHI9hO7kwJNFMuSylejFhMslxgdzGlIAyXauwg
```

With that variable set, we can use the `-H` flag with curl to set the `X-Tapis-Token` header as follows:

```
$ curl -H "X-Tapis-Token: $JWT" ....
```

Note also the *claims* associated with the access token. These claims provide information about the token, including the user it represents (`apitest` in the above example), the tenant it belongs to (`tacc` above) when it expires, etc. Tapis tokens always include the following standard claims:

Claim	Description	Example Value
sub	The subject of the token; the subject uniquely identifies the user in a Tapis installation. The format is `user @ tenant`	apitest@tacc
exp	The expiry associated with the token.	1595099456
jti	Unique identifier for the token.	f7b9b19e-6099-4f82-a722-b610c5d0ba0c
iss	The identifier (URL) of the issuer of the JWT. For Tapis, the issuer will be a Tokens API.	https://tacc.tapis.io/v3/tokens

Additional custom claims specific to Tapis are namespaced with `tapis/` at the beginning of the claim name. The authenticator for each tenant may optionally choose to support one or more of these additional claims. The following claims are encouraged and supported by the default OAuth2 Tapis authenticator.

Claim	Description	Example Value
tapis/tenant_id	The tenant of the subject.	tacc
tapis/username	The username of the subject.	apitest
tapis/token_type	Type of token: `access` or `refresh`	access
tapis/account_type	Type of account: `user` or `service`	user
tapis/delegation	Whether a delegation flow was used to generate this token. (`true` or `false`).	false
tapis/delegation_s	For a delegation token, the subject who actually generated the token. In form `user @ tenant`	supe-ruser@tacc
tapis/client_id	The id of the OAuth client used to generate the token.	tacc.CIC.tokenapp
tapis/grant_type	The grant type used to generate the token.	authoriza-tion_code

The authenticator for your tenant may include additional claims not listed here.

3.3.3 OAuth Clients

In order to use the more advanced OAuth2 flows, including any use of the authorization code grant type, the implicit grant type, and to generate refresh tokens with the password grant type, you must generate an OAuth2 *client*. Clients in OAuth2 represent applications (for example, a web or mobile application) that will interact with the OAuth2 server to generate tokens on behalf of one or more users. Clients are created and managed using the `/v3/oauth2/clients` endpoints.

Creating Clients

To create a client, make a POST request the the Clients API. All fields are optional; if you do not pass a `client_id` or `client_key` in the request, the clients API will generate random ones for you. In order to use the `authorize_code` and `implicit` grant types you will need to set the `callback_url` when registering your client (see *Authorization Code Grant - Generating Tokens For Users*). For a complete list of available parameters, see the API live-docs for Clients.

With PySDK:

```
>>> t.authenticator.create_client(client_id='test', callback_url='https://foo.example.
↪com/oauth2/callback')
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" -H "Content-type: application/json" -d '{"client_id":
↪"test", "callback_url": "https://foo.example.com/oauth2/callback"}' https://tacc.tapis.
↪io/v3/oauth2/clients
```

The response will be similar to

```
callback_url: https://foo.example.com/oauth2/callback
client_id: test
client_key: WQZlQlMoxOynW
create_time: Sat, 18 Jul 2020 19:09:47 GMT
description:
display_name: https://foo.example.com/oauth2/callback
last_update_time: Sat, 18 Jul 2020 19:09:47 GMT
owner: apitest
tenant_id: tacc
```

Listing Clients

With PySDK:

```
>>> t.authenticator.list_clients()
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/oauth2/clients
```

The response will be similar to

```
[
callback_url: https://foo.example.com/oauth2/callback
client_id: test
client_key: WQZlQlMoxOynW
create_time: Sat, 18 Jul 2020 19:09:47 GMT
description:
display_name: https://foo.example.com/oauth2/callback
last_update_time: Sat, 18 Jul 2020 19:09:47 GMT
owner: apitest
tenant_id: tacc]
```

Deleting Clients

You can also delete clients you are no longer using; just pass the `client_id` of the client to be deleted:

With PySDK:

```
>>> t.authenticator.delete_client(client_id='test')
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" -X DELETE https://tacc.tapis.io/v3/oauth2/clients/test
```

A null response is returned from a successful delete request.

3.3.4 Authorization Code Grant - Generating Tokens For Users

An important aspect of OAuth2 is that it enables applications to generate tokens on behalf of users without the applications needing to possess user credentials (i.e., passwords). In this section, we discuss using the OAuth2 *authorization code* grant type to do just that.

Assuming a Model-View-Controller (MVC) architecture, there are two controllers that must be written to support the authorization code grant type flow.

1. A controller to determine if the user already has a valid access token and direct them to the OAuth2 authorization server when they do not. This controller starts the authorization code process. To do so, it should:

- First inform the user that they will be asked to authenticate with their tenant username and password and then be asked to grant authorization to your client application.
- Redirect the user to the OAuth2 server's authorization URL. In the default Tapis authenticator, the authorization URL path is `/v3/oauth2/authorize`; for example, `https://tacc.tapis.io/v3/oauth2/authorize` in the `tacc` tenant.
- The redirect request should include the following query parameters:
  - `client_id`: the id of your client.
  - `redirect_uri`: the URI to redirect back to with the authorization code. This must match the `callback_url` parameter associated with your client.
  - `response_type`: should always have the value `code`.

2. A controller to process the authorization code returned and retrieve an access token on the user's behalf. This controller receives requests containing authorization codes from the OAuth2 server after the user has successfully authenticated with said OAuth2 server, and it immediately exchanges the code for a token.

- Responds to `GET` requests to the URL defined in the `callback_url` parameter of your client.
- Retrieves the `code` query parameter from the request.
- Makes a `POST` request to the OAuth2 server's tokens endpoint to generate a token. In the default Tapis authenticator, the tokens URL path is `/v3/oauth2/tokens`; for example, `https://tacc.tapis.io/v3/oauth2/tokens` in the `tacc` tenant. The POST body must include the following parameters:
  - `code`: the code the controller just received in the request from the OAuth2 server.
  - `redirect_uri`: should be the same as the `callback_url` parameter of your client.
  - `grant_type`: should always have the value `authorization_code`.

Note that many popular web frameworks support OAuth2 flows with minimal custom coding required.

The final step to using the authorization code grant type is to register a client (see above) with a `callback_url` parameter equal to the URL within your web application where it will handle converting authorization codes into access tokens (i.e., controller 2 above).

3.3.5 Implicit Grant - Directly Returning Tokens For Users

As mentioned in the authorization code grant section, OAuth2 provides an extra wall of security where the application does not encounter credentials of the user (i.e., passwords). In this section, we discuss using the OAuth2 *implicit* grant type.

Just like the authorization code grant type, the implicit grant type retrieves an access token on behalf of the user to login. In addition, the implicit grant type requires a redirect to the authorization server.

The difference between the two grant types is that the access token is directly delivered through the browser, rather than a code that is later exchanged for the access token. This allows the implicit grant type to be easier to set up, but much

less secure as it cannot authenticate the client. The typical use case for the implicit grant type would be for applications where the client secrets cannot be securely stored.

This is how to start the implicit flow:

- Redirect the user to the OAuth2 server's authorization URL. In the default Tapis authenticator, the authorization URL path is `/v3/oauth2/authorize`; for example, `https://tacc.tapis.io/v3/oauth2/authorize` in the `tacc` tenant.
- The redirect request should include the following query parameters:
  - `client_id`: the id of your client.
  - `redirect_uri`: the URI to redirect back to with the access token. This must match the `callback_url` parameter associated with your client.
  - `response_type`: should always have the value `token`.

Once the user completes the login process through the authorization server, the server redirects the user to the redirect uri along with the access token in the browser.

The final step is to collect the access token in the url and store it in the cookies.

The implicit grant type also requires a client (see above) with a `callback_url` parameter equal to the URL within your web application where it will handle storing the access tokens in the cookies.

Implicit Grant Tutorial

This section serves as a small tutorial to further understand how the implicit grant works and how to set it up.

The first step is to create an OAuth2 client as shown in the *OAuth Clients* section. While setting up the client, make sure to set the `callback_url` to the website that the user will be redirected to (`redirect_uri`) after successfully authenticating on the authorization server. The whole flow will not work if these two values are not identical. Additionally, make sure to remember the `client_id` as it will be needed later.

Note

The `callback_url` of the client created can be changed later on if needed, but the `client_id` cannot.

The next step is to redirect the user to the authorization server's website. The most common way is to create a button on the website that takes the user to the authorization server's website. Make sure to follow this specific formatting for the website url in order to prevent any errors from occuring:

```
https://{tenant}.tapis.io/v3/oauth2/login?client_id={client_id}&redirect_uri={redirect_
↪uri}&response_type={response_type}
```

This is the formatting that will be used for all OAuth2 authentication. Everything in curly brackets needs to be replace with the correct values to successfully set up the implicit flow. Make sure to drop the curly brackets when replacing them with their respective values.

- `{tenant}`: This is the tenant value. Examples are `tacc`, `scoped`, `dev`, etc.
- `{client_id}`: This is the client id for the client that was created in the first step.
- `{redirect_uri}`: This is the value that needs to be the same as the `callback_url`.

Important

The value that replaces this `{redirect_uri}` needs to be *URL Encoded*!! Use an online url encoder to help format the value if needed.

Example of a properly url encoded value:

```
From: https://tacc.tapis.io/v3/oauth2/login?
To:   https%3A%2F%2Ftacc.tapis.io%2Fv3%2Foauth2%2Flogin%3F
```

- `{response_type}`: This is the value that will be delivered through the browser and is dependent on the authorization type that is being used.
- For `implicit`, use `token`.
- For `authorization code`, use `code`.

Once the user has authenticated through the server, the user will automatically be redirected to the `redirect_uri` or `callback_url`. When redirected, the `access_token` (`code` for authorization code) will be returned in the url. Use a method to capture this value and store it in the cookies.

Important

Note that for both `access_token` and `code`, there are more parameters that are returned alongside these values through the url like `state`. Therefore, make sure to capture the value that is between the "*access_token*=" (or "*code*=") and "*&state*".

Finally, since the user does not need to physically see the redirect page, store the token in the cookies and redirect the user one last time to the main page.

Note

For the `authorization code` grant type, everything in this tutorial is pretty much identical. However, there is one more final step of exchanging the `code` returned through the url with the token to complete the flow.

3.3.6 The Tapis Token Web Application

Tapis provides a graphical interface via a web application that enables users to generate tokens. The Tapis Web Application is available by default for any tenant using the default Tapis authenticator, including the `tacc` tenant. The Tapis Token Web Application serves as an example of an application using the authorization code grant type.

The Tapis Token Web Application and its source code are available at the following URLs:

- Token App (`tacc` tenant): https://tacc.tapis.io/v3/oauth2/webapp
- Token App source code: https://github.com/tapis-project/authenticator